modelcontextprotocol / csharp-sdk

Code Issues 53 Pull requests 6 Discussions Actions Projects Security Insights

The official C# SDK for Model Context Protocol servers and clients. Maintained in collaboration with Microsoft.

www.nuget.org/packages/ModelContextProtocol

MIT license

Code of conduct

Security policy

2.7k stars 395 forks 53 watching Branches Activity Custom properties
Tags

Public repository

main 5 Branches 18 Tags Go to file t Go to file Add file Code

stephentoub Make CallToolResult.IsError optional (#573) ab51400 · 10 hours ago

.config	Update documentation configuration (#86)	3 months ago
.github/workflows	Make small corrections to release process docs...	last month
docs	Update documentation configuration (#86)	3 months ago
samples	Styling: remove a few redundant parens from ...	5 days ago
src	Make CallToolResult.IsError optional (#573)	10 hours ago
tests	Make CallToolResult.IsError optional (#573)	10 hours ago
.editorconfig	Wrap each request in a service scope (#276)	2 months ago
.gitattributes	Add gitattributes (#98)	3 months ago
.gitignore	Bump xunit.v3 to 2.0.0 (#231)	2 months ago
Directory.Build.props	Send MCP-Protocol-Version header in Streama...	2 weeks ago
Directory.Packages.props	Removed space (#563)	11 hours ago
LICENSE	Update copyright in LICENSE file and add THIR...	3 months ago
Makefile	Update documentation configuration (#86)	3 months ago
ModelContextProtocol.slnx	Add structured output/output schema support...	last month
Open.snk	Add strong naming (#49)	3 months ago
README.md	Styling: remove a few redundant parens from ...	5 days ago
SECURITY.md	Add SECURITY.md (#239)	2 months ago
THIRD-PARTY-NOTICES.txt	Add McpServer/ClientResource{Template} and ...	last month
global.json	Convert solution to slnx (#472)	last month
logo.png	Add Directory.Build.props, apply CPM, use NB....	3 months ago
nuget.config	Fix nuget config	3 months ago

README Code of conduct MIT license Security

MCP C# SDK

nuget v0.3.0-preview.1

The official C# SDK for the Model Context Protocol, enabling .NET applications, services, and libraries to implement and interact with MCP clients and servers. Please visit our API documentation for more details on available functionality.

Packages

This SDK consists of three main packages:

- ModelContextProtocol nuget v0.3.0-preview.1 - The main package with hosting and dependency injection extensions. This is the right fit for most projects that don't need HTTP server capabilities. This README serves as documentation for this package.
- ModelContextProtocol.AspNetCore nuget v0.3.0-preview.1 - The library for HTTP-based MCP servers. Documentation
- ModelContextProtocol.Core nuget v0.3.0-preview.1 - For people who only need to use the client or low-level server APIs and want the minimum number of dependencies. Documentation

> **Note**
>
> This project is in preview; breaking changes can be introduced without prior notice.

About MCP

The Model Context Protocol (MCP) is an open protocol that standardizes how applications provide context to Large Language Models (LLMs). It enables secure integration between LLMs and various data sources and tools.

For more information about MCP:

- Official Documentation
- Protocol Specification
- GitHub Organization

Installation

To get started, install the package from NuGet

```
dotnet add package ModelContextProtocol --prerelease
```

Getting Started (Client)

To get started writing a client, the `McpClientFactory.CreateAsync` method is used to instantiate and connect an `IMcpClient` to a server. Once you have an `IMcpClient`, you can interact with it, such as to enumerate all available tools and invoke tools.

```
var clientTransport = new StdioClientTransport(new StdioClientTransportOptions
{
    Name = "Everything",
    Command = "npx",
    Arguments = ["-y", "@modelcontextprotocol/server-everything"],
});

var client = await McpClientFactory.CreateAsync(clientTransport);
```

```
// Print the list of tools available from the server.
foreach (var tool in await client.ListToolsAsync())
{
    Console.WriteLine($"{tool.Name} ({tool.Description})");
}

// Execute a tool (this would normally be driven by LLM tool invocations).
var result = await client.CallToolAsync(
    "echo",
    new Dictionary<string, object?>() { ["message"] = "Hello MCP!" },
    cancellationToken:CancellationToken.None);

// echo always returns one and only one text content object
Console.WriteLine(result.Content.First(c => c.Type == "text").Text);
```

You can find samples demonstrating how to use ModelContextProtocol with an LLM SDK in the samples directory, and also refer to the tests project for more examples. Additional examples and documentation will be added as in the near future.

Clients can connect to any MCP server, not just ones created using this library. The protocol is designed to be server-agnostic, so you can use this library to connect to any compliant server.

Tools can be easily exposed for immediate use by `IChatClient`s, because `McpClientTool` inherits from `AIFunction`.

```
// Get available functions.
IList<McpClientTool> tools = await client.ListToolsAsync();

// Call the chat client using the tools.
IChatClient chatClient = ...;
var response = await chatClient.GetResponseAsync(
    "your prompt here",
    new() { Tools = [.. tools] },
```

Getting Started (Server)

Here is an example of how to create an MCP server and register all tools from the current application. It includes a simple echo tool as an example (this is included in the same file here for easy of copy and paste, but it needn't be in the same file... the employed overload of `WithTools` examines the current assembly for classes with the `McpServerToolType` attribute, and registers all methods with the `McpTool` attribute as tools.)

```
dotnet add package ModelContextProtocol --prerelease
dotnet add package Microsoft.Extensions.Hosting
```

```
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Logging;
using ModelContextProtocol.Server;
using System.ComponentModel;

var builder = Host.CreateApplicationBuilder(args);
builder.Logging.AddConsole(consoleLogOptions =>
{
    // Configure all logs to go to stderr
    consoleLogOptions.LogToStandardErrorThreshold = LogLevel.Trace;
});
builder.Services
    .AddMcpServer()
    .WithStdioServerTransport()
    .WithToolsFromAssembly();
await builder.Build().RunAsync();

[McpServerToolType]
public static class EchoTool
{
    [McpServerTool, Description("Echoes the message back to the client.")]
```

```
    public static string Echo(string message) => $"hello {message}";
}
```

Tools can have the `IMcpServer` representing the server injected via a parameter to the method, and can use that for interaction with the connected client. Similarly, arguments may be injected via dependency injection. For example, this tool will use the supplied `IMcpServer` to make sampling requests back to the client in order to summarize content it downloads from the specified url via an `HttpClient` injected via dependency injection.

```
[McpServerTool(Name = "SummarizeContentFromUrl"), Description("Summarizes content downloaded from a specific URI")]
public static async Task<string> SummarizeDownloadedContent(
    IMcpServer thisServer,
    HttpClient httpClient,
    [Description("The url from which to download the content to summarize")] string url,
    CancellationToken cancellationToken)
{
    string content = await httpClient.GetStringAsync(url);

    ChatMessage[] messages =
    [
        new(ChatRole.User, "Briefly summarize the following downloaded content:"),
        new(ChatRole.User, content),
    ];

    ChatOptions options = new()
    {
        MaxOutputTokens = 256,
        Temperature = 0.3f,
    };

    return $"Summary: {await thisServer.AsSamplingChatClient().GetResponseAsync(messages, options, cancellationToken)}";
}
```

Prompts can be exposed in a similar manner, using `[McpServerPrompt]`, e.g.

```
[McpServerPromptType]
public static class MyPrompts
{
    [McpServerPrompt, Description("Creates a prompt to summarize the provided message.")]
    public static ChatMessage Summarize([Description("The content to summarize")] string content) =>
        new(ChatRole.User, $"Please summarize this content into a single sentence: {content}");
}
```

More control is also available, with fine-grained control over configuring the server and how it should handle client requests. For example:

```
using ModelContextProtocol;
using ModelContextProtocol.Protocol;
using ModelContextProtocol.Server;
using System.Text.Json;

McpServerOptions options = new()
{
    ServerInfo = new Implementation { Name = "MyServer", Version = "1.0.0" },
    Capabilities = new ServerCapabilities
    {
        Tools = new ToolsCapability
        {
            ListToolsHandler = (request, cancellationToken) =>
                ValueTask.FromResult(new ListToolsResult
                {
                    Tools =
                    [
                        new Tool
                        {
                            Name = "echo",
                            Description = "Echoes the input back to the client.",
```

```
                        InputSchema = JsonSerializer.Deserialize<JsonElement>("""
                            {
                                "type": "object",
                                "properties": {
                                  "message": {
                                    "type": "string",
                                    "description": "The input to echo back"
                                  }
                                },
                                "required": ["message"]
                            }
                            """),
                    }
                ]
            }),

        CallToolHandler = (request, cancellationToken) =>
        {
            if (request.Params?.Name == "echo")
            {
                if (request.Params.Arguments?.TryGetValue("message", out var message) is not true)
                {
                    throw new McpException("Missing required argument 'message'");
                }

                return ValueTask.FromResult(new CallToolResult
                {
                    Content = [new TextContentBlock { Text = $"Echo: {message}", Type = "text" }]
                });
            }

            throw new McpException($"Unknown tool: '{request.Params?.Name}'");
        },
    }
  },
};

await using IMcpServer server = McpServerFactory.Create(new StdioServerTransport("MyServer"), options);
await server.RunAsync();
```

Acknowledgements

The starting point for this library was a project called mcpdotnet, initiated by Peder Holdgaard Pedersen. We are grateful for the work done by Peder and other contributors to that repository, which created a solid foundation for this library.

License

This project is licensed under the MIT License.

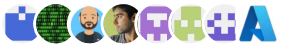
Releases

18 tags

Packages

No packages published

Used by 744



Contributors 41

+ 27 contributors

Deployments 24

github-pages last week

+ 23 deployments

Languages

C# 99.9% Makefile 0.1%